

React Javascript

for web → react-dom

for app → react-native

Only react way (bulky): Basic → (1)

npm → node packet manager

npm → node packet executor

↳ npm create-react-app projectname

entry point of project → package.json

for project running → run start script (in package.json).

→ npm run start

→ npm run build → convert react to simple one

↳ translate, bundle ^{js} & minify

Vite + React: (not bulky way) → (2)

npm way to create is too bulky.

so use vite way to make project.

↓
bundler

→ npm create vite@latest → (i)

↳ then select (name, framework, variant).

minimal usable dependencies.

as no node_modules folder in vite way.

→ npm install → (ii)

→ npm run dev → (iii)

Basic React one :

src folder : → to make simpler for beginn

delete setupTests.js

" reportWebVitals

" logo.svg

" App.test

" App.css

" index.css (will use later).

"Remember! they aren't useless, for someone making react for the first time, we are deleting to make the src folder simpler.

→ now remove imports.

Now :-

App.js:

code snippet

```
function App() {  
  return (  
    <h1>Hello world</h1>  
  );  
}  
export default App;
```

Now import in "index.js"

→ import App from './App';

Index.js :

```
import _____  
import _____  
import App from './App'
```

```
const root = _____
```

```
root.render(  
  <React.StrictMode>  
    <App />  
  <React.StrictMode>  
);
```

used here
skip other lines
for now, will
deep dive later

→ now npm run dev

*
→ Congrats , own first react project.*

Now Vite + React one :

src folder :

delete assets (folder)

" App.css

" index.css

Now remains:

App.jsx , main.jsx

After cleanup:

// names of files can be
your choice.

App.jsx: same code as in App.js → prev page

main.jsx: " " " " index.js → prev page

→ now

npm run dev

Check
Point

We learnt how to make react project
(2 ways) (vite + react preferred), and
clean up project to make it simpler.

Understanding react flow & structure

Foundation Rule:

Whenever we use a js file for html file → its compulsory to import the js file into html one.

1). Basic React flow:

→ All the dependencies in our "package.json" folder are in "node_modules" folder after installation.

/Public folder:

→ index.html:

: remove all comments first.

<body>

<noscript> You need to enable js to run this app. </noscript>

<div id="root"></div>

</body>

if a browser does not have js enable
Go show this
msg.

Src folder:

→ index.js:

to manipulate
web dom

import React from 'react';

import ReactDOM from 'react-dom/client';

foundational
core library

→ implementation
on web.

```
const root = ReactDOM.createRoot(  
  document.getElementById('root')  
);
```

Imp point:

* a website has its own DOM but react also makes it own DOM called 'Virtual DOM'.

It compares virtual DOM with browser's DOM and see what's the difference and changes only what's needs to be changed.

— ReactDOM.createRoot (.....) takes element

• creates root but where?

In index.html there's a div which has id='root':

→ const root = reference of that is stored in root variable.

```
now root.render( —————  
                  <App />  
                  );
```

↳ the strictMode isn't necessary

App came from the App.js file.

React said:

"take a function, return html in that function and I'll render that html."

* Basically, it's making us write js through html.*

Q: Now, remember the foundation rule? we didn't import script file into html so how does it know which file & how to run?

In package.json theres "read-scripts" it automatically loads javascript into index.html.

Now vite one + React:

In this package.json in dependencies there's no "react-scripts".

• So here in /index.html:

they imported the script.

→ `<script type="module" src="/....."></script>`

main.jsx:

just didn't create a root variable.

used "." directly bcoz all working is basically same.

Q: I made a function in ~~App~~ App.jsx
gave it to main.jsx to render.
But can I directly render it in
App.jsx?

→ go to src

→ create new file (anyname.jsx)

inside create a function. ^{anyname}

{ [} }

```
function Reactfunc() {  
  return (  
    <h3> hello guys </h3>  
  )  
}  
→ export default reactfunc
```

→ now we can send it to main.jsx &
App.jsx both of them can use it.

1) App.jsx:

```
import anyname from "  
import Reactfunc from "./anyname"  
function App() {  
  return (  
    <Reactfunc />  
  )  
}
```

export default App.

It runs.

Some practices: in vite as it is used mostly

→ Always use .jsx instead of .js when creating a file.

→ The function name must always start with a capital letter.

→ same in basic react.

reactfunc() { } X

Reactfunc() { } ✓

★ "A function can only return one thing"
you noticed? we deleted the already
written html and replaced it with our
<Reactfunc/>".★

if we returned;

```
function App() {
```

```
  return (
```

```
    <Reactfunc/>
```

```
    <h1> whatever </h1>
```

```
  )
```

```
}
```

} gives error

We can solve by two ways.

1) . Wrap in `<div>` :

```
return ( <div>  
    <Reactfunc />  
    <h3>... --</h3>  
  </div>  
)
```

or

2) . `Fragment(<>)`:

```
return (  
  <>  
    <Reactfunc />  
    <h3>...--</h3>  
  </>  
) .
```

* Checkpoint we learnt best practice for filename , function names & in vite we use .jsx only .js can be used in basic react . & Virtual DOM know how & etc.*

Create your own library & JSX

Index.html:

Same as before.

~~const root =~~ `<div id='root'></div>`

~~<div id='root'>~~ `<script src='./Customreact.js'></script>`

Customreact.js:-

makes a tree.

when a tag goes to react for render it looks like this:

- 1). `const reactElement = {`
- 2). `type:`
- 3). `props: {`
- 4). `href: 'https://google.com',`
- 5). `target: '_blank'`
- 6). `},`
- 7). `children: 'Click to visit google'`
- 8). `}`

10). `const mainCont = document.getElementById('root')`

12). `customRender(reactElement, mainCont)`

13). `function customRender(reactElement, mainCont)`

14). `{ const DOMElement = document.createElement`
`(reactElement.type)`

```

15) domElement.innerHTML = reactElement.children
16) domElement.setAttribute('href', reactElement.props.href)
17) " " ['target', ... " " target]
18) mainContainer.appendChild(domElement)
    }

```

There's a problem in this code, if there was one DOMElement attribute or three, it would be a problem.

↳ 13 to 18th line

Now we'll use loop based:

```

13) function customRender(reactElement, container)
14) { const domElement = document.createElement
15) (reactElement.type)
16) domElement.innerHTML = reactElement.children
17) for(const prop in reactElement props) {
18) if(prop === children) continue;
19) domElement.setAttribute(prop, reactElement
20) .props[prop])
21) }
22) container.appendChild(Child domElement)
23) }

```

React.createElement:

```
const reactElement = React.createElement(  
  → 'a', // tag 1st parameter  
  { href: 'https://google.com' // obj 2nd  
    target: '_blank' },  
  → 'click to visit google' // direct text 3rd  
)  
  
{ ReactDOM.createRoot( . . . . . ).  
  render (  
    reactElement ) ; /works
```

App.jsx: using variable inside component

```
function App() { const username = "hu3"  
  return (  
    <>  
      <h1> yo {username} </h1>  
      <p> nice </p>  
    </>  
  )  
}
```

* Can't do evaluated expression inside these curly brackets *

Where variables go in the react tree?

* These variables go to as like 4th parameter in the `React.createElement` *

`React.createElement (tag, object, text, variables)`

Why {} have no evaluated expression?

* So react converts the function to object as objects doesn't have if else or evaluate that's why it doesn't have in curly bracket.*

Checkpoint:

- Made our own react Element. (custom)

↳ basic & loop (2 approach).

- `React.createElement()`, how it works & expects. in parameters.

Why we need Hooks & Projects

Make a new project. We are making a simple counter.

→ make vite+React project.

App.jsx:-

```
function App() {
```

```
  let counter = 15
```

```
  const addValue = () => {
```

```
    console.log("clicked", Math.random());
```

```
    counter = counter + 1;
```

```
  }
```

```
  return (
```

```
    <>
```

```
      <h1> First Small Project </h1>
```

```
      <h2> Counter </h2>
```

```
      <button onClick = {addValue}>
```

```
        Add value </button>
```

```
      <br/>
```

```
      <button> remove value </button>
```

```
    </>
```

You noticed?

* When you click the button, it works but the counter is not updating in the browser page.*

→ Counter is updating, the problem is UI updation.

→ That's where hooks come ?:

React said you do whatever you want to but in case of updating UI i'll do my own thing.

So react gave us methods to do UI updation, they are called "hooks".

→ React controls UI updation, in a nutshell.

→ How to use hook?

We have to import hooks as they are functions just like App.

```
import {useState} from 'react'
```



name can be anything

makes let

counter behind function
the scenes.

counter def value is 15.

function App() {
error ~~const~~ let [counter, setCounter] = useState(15)

```
const addvalue = () => {  
  console.log("clicked", counter);  
  setCounter(counter + 1);  
}  
return ( same as before )  
};
```

make remove yourself (Task)

Also it shouldn't go below 0 & above 20.
_____.

Virtual DOM, fibre & reconciliation

* Virtual DOM isn't used now in React *

createRoot: creates a DOM like structure as it compares it's dom with main DOM & only update which is updated in UI.

→ but browser removes full DOM & re paints it (page reload).

→ react-fibre-architecture (github repo).
↳ read this REPO.

There is virtual DOM but the algo used to update that is fibre.

Reconciliation: Algo to reconsider which to update and which to not.

↳ diff one tree with another (the browser & virtual DOM one).

→ when setState is used a new tree used & diffed with browser DOM & only needed updates are done.

* In fibre if you wanna improve lists' algos then at each iteration you must use keys.*

Keypoints:

- In UI, its not necessary for every update to be applied immediately, doing so can be wasteful, causing frames to drop. & degrade user exp.

- Diff types of updates have diff priorities - an animation update more quickly than say, an update from the store.

- Push based approach (you decide how to schedule work).

- Pull based approach (React makes decisions for you).

Tailwind & props in react

Tailwind+Vite setup:

1). • npm install -D tailwindcss postcss autoprefixer
↳ to install / tailwind dependencies

• dev dependencies & dependencies

2). • npx tailwindcss init -p
↳ tailwind.config.js is created
file

```
export default {  
  content: [  
    "./index.html",  
    "src/**/*.{js,ts,jsx,tsx}",  
  ],  
  theme: {  
    extend: {},  
  },  
  plugins: [],  
}
```

content: tell Tailwind:

"Scan these files & generate only the classes being used"

means:

- /index.html → scan it
 - /src/ ,tsx} → scan inside src
- if Tailwind classes are not inside content, CSS will not work.

theme:

used to customize design values like:

- color
- breakpoints
- fonts
- animations
- spacing

e.g.:

```
theme: {  
  colors: {  
    primary: "#1e...",  
  },  
  spacing: {  
    128: "32rem",  
  },  
},
```

extend means keep tailwind defaults & add your own values.

plugins:

add extra tailwind features

```
[ plugins: [  
    require('@tailwindcss/forms')  
  ]
```

add better styling utilities for forms.

Now,

go to "index.css" in src folder
and add:

- @tailwind base;
- @tailwind components;
- @tailwind utilities;

Now, go to App.jsx to test tailwind

```
function App() {  
  return (  
    <>
```

w/o tailwind <h1> Tailwind test </h1>

w tailwind → <h1 class="bg-green-400
text-black

p-4

rounded-xl

Devui.io → To get components (pre built)

Props: makes component reusable.
html, css, js at one place.

to src → component folder (make new)
react snippet → extension → ~~rfce~~ rfc

make Card.jsx (file):

```
import React from 'react'

function Card() {
  return (
    <div>Card</div> or full card code
  )
}

export default Card
```

Now to App.jsx:

```
import Card from './components/Card'
```

```
return (
```

```
<Card /> <Card />
```

→ use as many times as you want

- Why Props:

So, whenever you use '`<Card />`' it will have same details name or everything.

So we use props as parameters to get the needed information.

- `function Card(props) { ... }`
 ↓
 in it is empty

- Anything you write in `<Card ... />`
 ↑
 goes in props.

→ `<Card channel="chai aur code" />`

props object will have this.

Object pass,

to pass object we can't directly pass that we have to separately create the object.

making obj:

```
let myObj = {  
  username: "huzairfa",  
  age: 21  
}
```

Passing:

<Card someObj = '{myObj}' / >

- Same way Array can be also passed.

Using of Props:

→ function Card(props) { ... }
 console.log(props.username)

or

destructuring can also be done.

→ function Card({username}) {
 console.log(username) }

* Task

change card content now diff for two three cards.

if someone doesn't pass value.

→ { btnText || "visit me" }

or

→ Card({username, btnText = "visit me"}

Foundations Completed

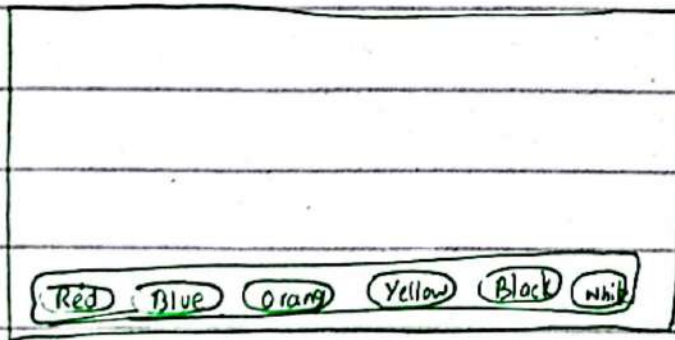
Interview Q:

```
const addValue = () => {  
  setCounter((prevCounter) => prevCounter + 1)  
  setCounter((counter) => counter + 1)  
}
```

use same [↑] though for consistency

First Project in React:

Colors changer



Whatever button you touch screen should be of that color.